# Luciteria Collector Cabinet — Assignment Engine Logic

## Overview

The Assignment Engine determines which product to send a subscriber in their next shipment. Its primary goals are:

1. **Preventing duplicates** — never send what they already own
2. **Collection type filtering** — only assign products matching the customer's collection type
3. **Precious metal exclusion** — Re, Rh, Au, Os, Ru, Pd, Ir, Pt excluded from subscription (Ag allowed)
4. **Discount tracking** — calculate and flag subscription vs. retail price discounts
5. **Sequence preview** — show the next 4+ months of planned assignments
6. **Out-of-stock shifting** — automatically substitute when planned items become unavailable

## Core Algorithm (Phase 2)

```
FUNCTION assignNextItem(customer, strategy):

  INPUT:
    - customer: Customer record (with collectionType)
    - ownedProductIds[]: Products customer already owns
    - shippedProductIds[]: Products previously sent via subscription
    - preferences: Customer preference settings
    - wishlistProductIds[]: Products on customer's wishlist
    - allProducts[]: Full product catalog with current inventory
    - strategy: Assignment strategy to use

  OUTPUT:
    - { success, product, reason, requiresManualReview, flags, alternates,
        exception, discountPercent, retailPrice, subscriptionCost }

  ═══ STEP 1: Build Initial Candidate Pool ═══

  candidates = allProducts
    .WHERE status = "Active"
    .WHERE inventoryQty > 0
    .WHERE collectionType = customer.collectionType    // NEW Phase 2
    .EXCLUDE precious metals (Re, Rh, Au, Os, Ru, Pd, Ir, Pt)  // NEW Phase 2

  IF candidates.length == 0:
    RETURN failure("No products in stock for this collection type")
    → Route to exception queue

  ═══ STEP 2: Apply Duplicate Handling ═══

  (Same as Phase 1 — see modes below)

  ═══ STEP 3: Apply Preference Filters ═══

  (Same as Phase 1 — category, format, budget filters)

  ═══ STEP 4: Check Remaining Candidates ═══

  IF candidates.length == 0:
    RETURN failure("No eligible items after applying preferences")
    → Route to exception queue

  ═══ STEP 5: Rank by Strategy ═══

  (Same 5 strategies as Phase 1)

  ═══ STEP 6: Select & Calculate Discount ═══  // NEW Phase 2

  selected = candidates[0]
  alternates = candidates[1..3]

  // Calculate discount
  retailPrice = selected.retailPrice || selected.priceUsd
  subscriptionCost = selected.subscriptionCost || selected.priceUsd
  discountPercent = ((retailPrice - subscriptionCost) / retailPrice) * 100

  ═══ STEP 7: Flag for Review ═══  // ENHANCED Phase 2

  flags = []
  IF selected.priceUsd > 200: flags.ADD("high_value")
  IF selected.inventoryQty <= 2: flags.ADD("low_stock")
  IF selected.rarityTier IN ["legendary", "ultra-rare"]: flags.ADD("rare_item")
  IF discountPercent > 20: flags.ADD("high_discount")           // NEW
```

```
  IF discountPercent > 20: triggerAdminAlert(customer, selected) // NEW

  RETURN {
    success: true,
    product: selected,
    reason: "Selected via {strategy} strategy",
    requiresManualReview: flags.length > 0,
    flags,
    alternates,
    discountPercent,     // NEW
    retailPrice,         // NEW
    subscriptionCost,    // NEW
  }
```

## Sequence Preview (Phase 2)

```
FUNCTION previewSequence(customer, monthsAhead = 4):

  // Simulates future assignments without committing
  simulatedOwned = [...customer.ownedProductIds]
  sequence = []

  FOR month = 1 TO monthsAhead:
    result = assignNextItem(customer, strategy, simulatedOwned)
    IF result.success:
      sequence.ADD({
        month: month,
        product: result.product,
        discountPercent: result.discountPercent
      })
      simulatedOwned.ADD(result.product.id)  // Simulate ownership
    ELSE:
      sequence.ADD({ month: month, product: null, reason: result.reason })

  RETURN sequence
```

## Out-of-Stock Shifting (Phase 2)

```
When a planned item goes out of stock before shipment:
1. Engine detects OOS during pre-shipment check
2. Automatically re-runs assignment for that customer
3. Skips the OOS product (inventoryQty = 0 filtered in Step 1)
4. Selects next best candidate
5. Logs the shift with reason
6. Notifies admin of the change
```

## Precious Metal Exclusion (Phase 2)

```
EXCLUDED_SYMBOLS = ['Re', 'Rh', 'Au', 'Os', 'Ru', 'Pd', 'Ir', 'Pt']
// Silver (Ag) is explicitly ALLOWED in subscription

These elements are:
- Excluded from automatic subscription assignments
- Still available for direct purchase
- Shown in periodic table UI (marked as "Not in subscription")
- Reason: Price volatility and high cost makes them unsuitable for fixed-price sub-
scription
```

## Collection Type Filtering (Phase 2)

```
Collection Types:
  - "10mm"     → 10mm density cubes
  - "25.4mm"   → 1-inch density cubes
  - "50mm"     → 50mm display cubes
  - "lucite"   → Lucite-embedded specimens
  - "ampoules" → Sealed glass ampoules

Rules:
  - Customer selects type during onboarding (locked in Phase 2)
  - Assignment engine only considers products matching customer's type
  - Products tagged with collectionType in database
  - Future: multi-collection support (Phase 3+)
```

## Discount Tracking (Phase 2)

```
Each product has:
  - retailPrice: Full retail price
  - subscriptionCost: Subscription price (typically lower)
  - discountPercent = (retail - subscription) / retail × 100

Admin Alerts:
  - >20% discount: Red flag in admin dashboard
  - Discount tracked per shipment in history
  - Admin can see discount breakdown in operations view

Grandfathered Pricing:
  - Customers who subscribed before a price increase keep their old price
  - Grandfathering clock pauses during subscription pause
  - grandPriceAtLock stored on customer and subscription records
```

## Assignment Strategies

| Strategy | Description | Best For |
|---|---|---|
| **wishlist_priority** | Wishlist items first, then by atomic number | Default — respects customer intent |
| **oldest_missing** | Lowest atomic number missing | Completionists building in order |
| **highest_overstock** | Items with most inventory | Inventory management optimization |
| **category_rotation** | Rotate through categories | Variety-seeking collectors |
| **surprise** | Random eligible item | “Surprise me” mode |

## Duplicate Handling Modes

| Mode | Behavior | Use Case |
|---|---|---|
| **never** | Exclude owned + previously shipped | Purist collectors |
| **missing_only** | Same as never (strictest) | Default safe mode |
| **curated_subs** | Exclude owned, allow re-ships | Different format exploration |
| **allow_if_limited** | Try no-dupes first, fall back | When catalog is nearly complete |
| **surprise** | No duplicate filtering | Trust-based, adventurous |

## Admin Manual Override (Phase 2)

```
Admin can:
1. Override engine's recommendation with manual product selection
2. Override is validated: product must be in stock, match collection type
3. Override logged with admin name and timestamp
4. Customer notified of final assignment
5. Discount still calculated and tracked for overridden assignments
```

# Edge Cases

## Customer Owns Everything in Their Collection Type

```
Result: No eligible items after Step 1 (collection type filter)
Action: Exception created → admin options:
  - Suggest switching collection type
  - Offer premium/limited edition items
  - Pause subscription with grandfathering preserved
```

## Out-of-Stock Wishlist Item

```
Result: Engine skips OOS item, selects next eligible
Action: Customer receives restock notification (via notification stub)
```

## 5-Day Billing Edge Case

```
Scenario: Customer signs up Jan 28
Result: First element ships immediately, next billing Feb 1 (not Feb 28)
Reason: Avoids double-charge within 5 days
```

## High Discount Alert (>20%)

```
Result: Assignment flagged for admin review
Action: Admin sees red discount badge in operations dashboard
Decision: Approve (accept discount) or substitute with lower-discount item
```

# Implementation

The assignment engine is implemented in `app/lib/assignment-engine.server.js` as a set of pure functions. Key exports:

- `assignNextItem(customerId)` — Core assignment with collection type filtering
- `previewSequence(customerId, months)` — Preview next N months of assignments
- `manualOverride(customerId, productId, adminName)` — Admin override
- `getDiscountInfo(product)` — Calculate discount percentage

The Admin Operations page runs the engine for each upcoming subscription and displays results with discount monitoring and sequence preview.